

COMP3026E Distributed Systems

7. Replication State Machine

James Jianqiao Yu

Harbin Institute of Technology, Shenzhen



From Eventual to Strong Consistency



- Eventual consistency
 - **Multi-master**: Any node can accept operation
 - Asynchronously, nodes synchronize state



From Eventual to Strong Consistency

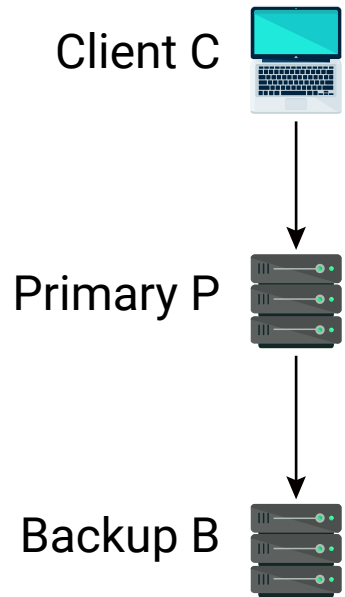


- Eventual consistency
 - **Multi-master**: Any node can accept operation
 - Asynchronously, nodes synchronize state
- Eventual consistency inappropriate for many applications
 - Imagine bank ledger as eventually consistent

From Eventual to Strong Consistency

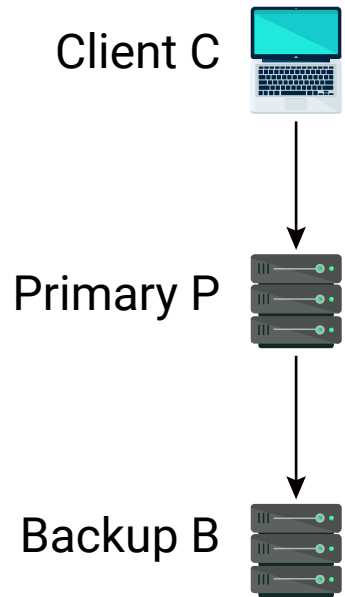
- Eventual consistency
 - **Multi-master**: Any node can accept operation
 - Asynchronously, nodes synchronize state
- Eventual consistency inappropriate for many applications
 - Imagine bank ledger as eventually consistent
- Stronger consistency makes applications **easier to write**
 - (More on downsides later)

Primary-Backup Replication



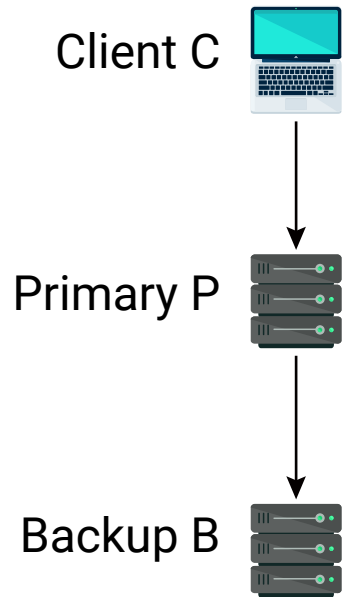
- **Mechanism:** Replicate and separate servers
- **Goal #1:** Provide a highly reliable service (despite failures)
- **Goal #2:** Servers should behave just like a single, more reliable server

Primary-Backup Replication



- Nominate one replica **primary**, other **backup**
 - Clients send all operations to current **primary**
 - Primary **orders** clients' operations
- Only **one primary** at a time

Primary-Backup Replication



- Nominate one replica **primary**, other **backup**
 - Clients send all operations to current **primary**
 - Primary **orders** clients' operations
- Only **one primary** at a time

Need to keep clients, primary, and backup in sync:
who is primary and who is backup

State Machine Replication



- Idea: A **replica** is essentially a state machine
 - Set of `<key, value>` pairs is **state**
 - Operations **transition** between states

State Machine Replication

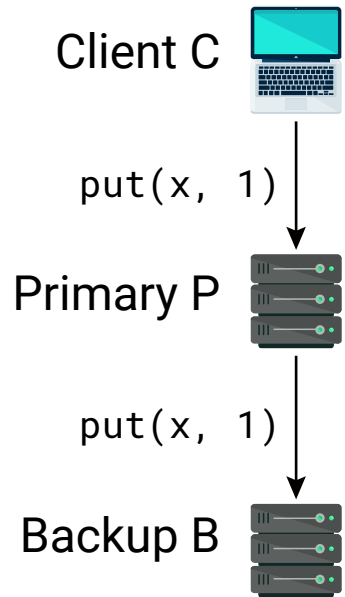
- Idea: A **replica** is essentially a state machine
 - Set of `<key, value>` pairs is **state**
 - Operations **transition** between states
- Need an operation to be executed on all replicas, or none at all
 - i.e., we need distributed **all-or-nothing atomicity**
 - If operation is deterministic, replicas will end in same state

State Machine Replication

- Idea: A **replica** is essentially a state machine
 - Set of `<key, value>` pairs is **state**
 - Operations **transition** between states
- Need an operation to be executed on all replicas, or none at all
 - i.e., we need distributed **all-or-nothing atomicity**
 - If operation is deterministic, replicas will end in same state

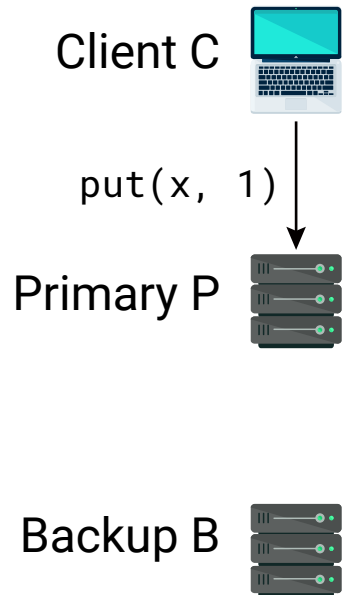
Key assumption Operations are **deterministic**

Primary-Backup Replication



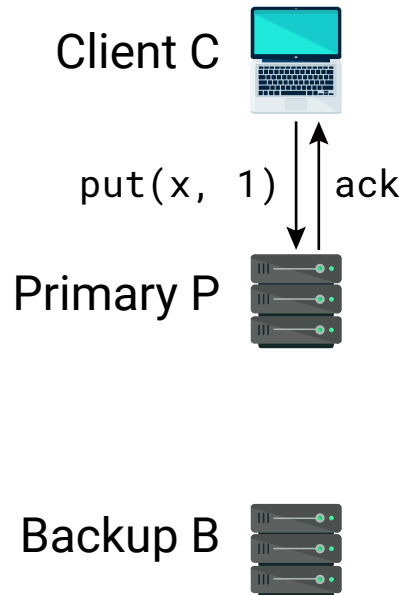
1. Primary gets operations
2. Primary orders operations into log
3. Replicates log of operations to backup
4. Backup executes operations or writes to log

Asynchronous Primary-Backup Replication



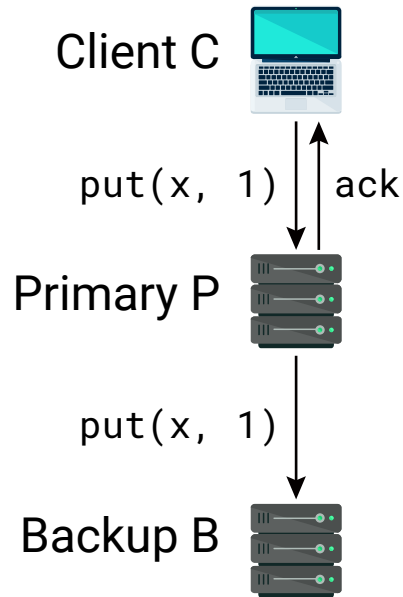
1. Primary gets operations
2. Primary **executes operations**
3. Primary orders operations into log
4. Replicates log of operations to backup
5. Backup executes operations or writes to log

Asynchronous Primary-Backup Replication



1. Primary gets operations
2. Primary **executes operations**
3. Primary orders operations into log
4. Replicates log of operations to backup
5. Backup executes operations or writes to log

Asynchronous Primary-Backup Replication



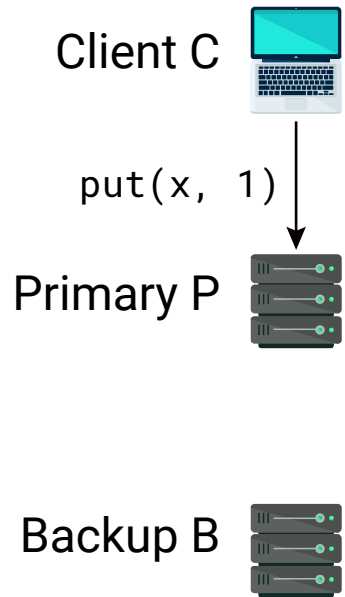
1. Primary gets operations
2. Primary **executes operations**
3. Primary orders operations into log
4. Replicates log of operations to backup
5. Backup executes operations or writes to log

Why Go / No Go Asynchronous?

- One word: **Latency**
 - Asynchronous replication removes the network “round-trip” from the critical path of the client’s request
- What happens if the Primary crashes **after** responding but **before** log entry reaches the Backup?
 - Client think the data is safe
 - The only server knows the data is lost
 - Backup will be missing the log entry, and thus the state change
 - Strong Consistency is **violated**



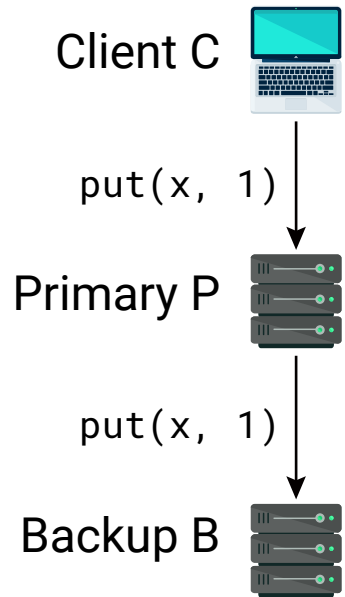
Synchronous Primary-Backup Replication



1. Primary gets operations
2. Primary orders operations into log
3. Replicates log of operations to backup
4. Backup executes operations or writes to log
5. Primary gets **ack**, **executes operations**



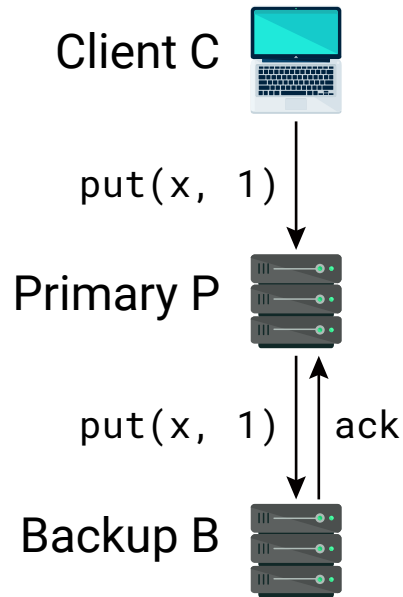
Synchronous Primary-Backup Replication



1. Primary gets operations
2. Primary orders operations into log
3. Replicates log of operations to backup
4. Backup executes operations or writes to log
5. Primary gets **ack**, **executes operations**



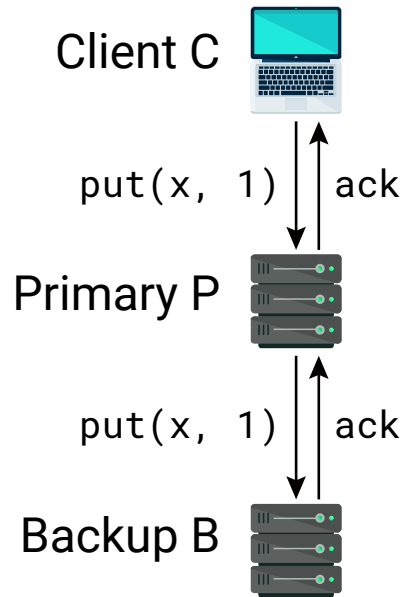
Synchronous Primary-Backup Replication



1. Primary gets operations
2. Primary orders operations into log
3. Replicates log of operations to backup
4. Backup executes operations or writes to log
5. Primary gets **ack**, **executes operations**



Synchronous Primary-Backup Replication



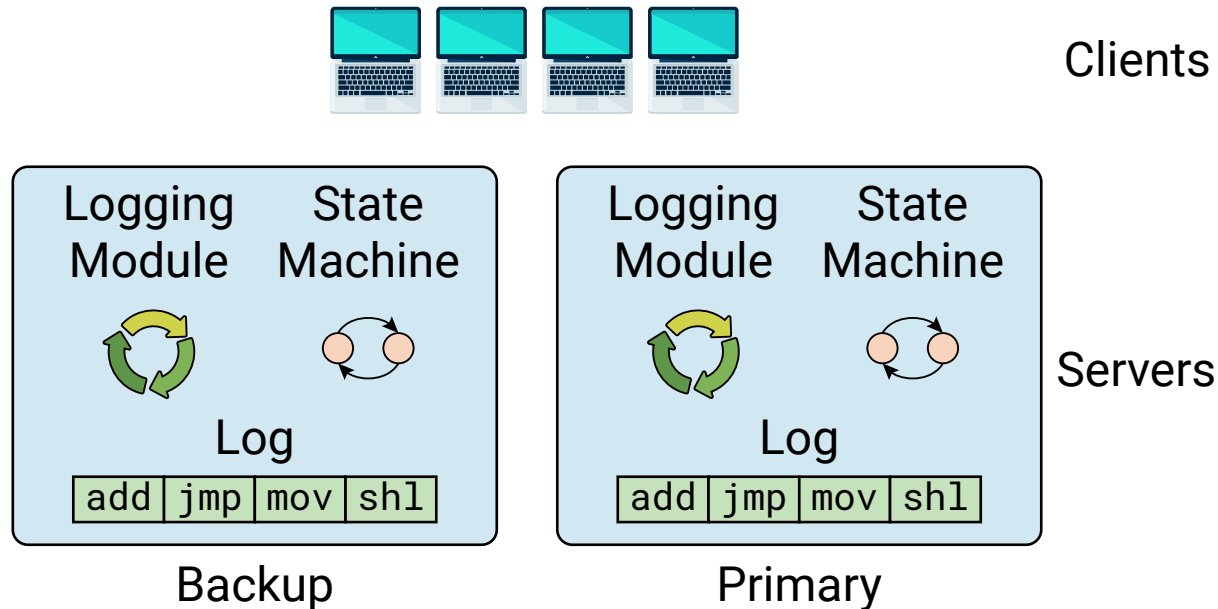
1. Primary gets operations
2. Primary orders operations into log
3. Replicates log of operations to backup
4. Backup executes operations or writes to log
5. Primary gets **ack**, **executes operations**

Asynchronous vs Synchronous Replication

- **Asynchronous:** High performance, low durability
 - Best for non-critical “fire and forget” data
- **Synchronous:** Lower performance, maximum durability
 - Essential for state that must never be lost.
- Semi-Synchronous more common in practice
 - More in next chapter

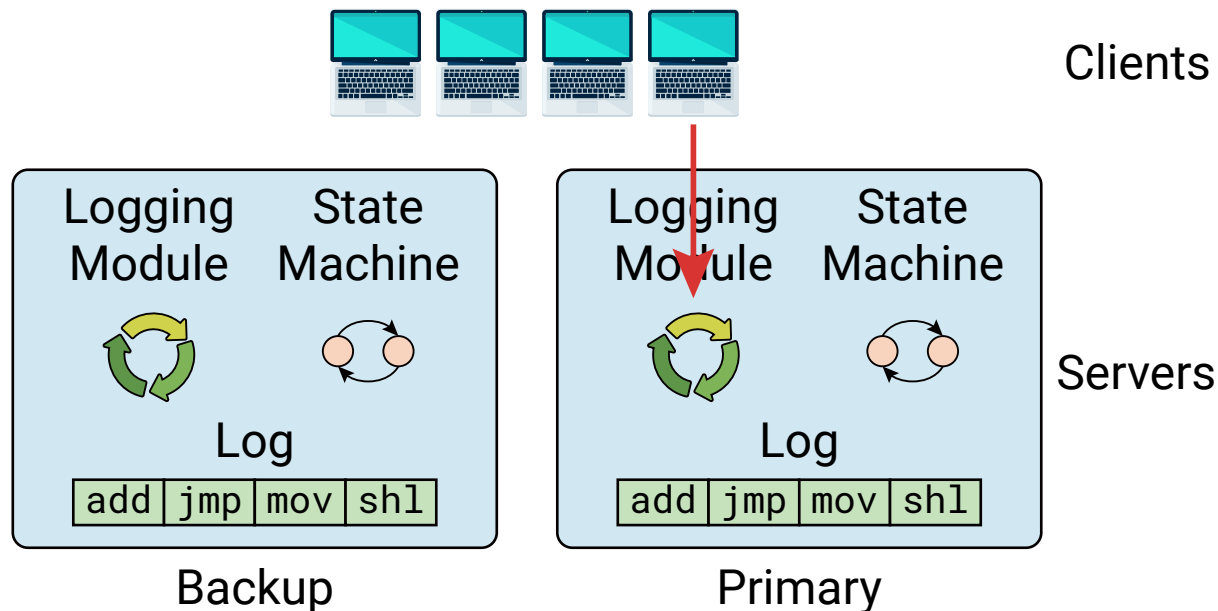
Keep in Mind The log is our “Source of Truth” for the state of the system

Why Does This Work?



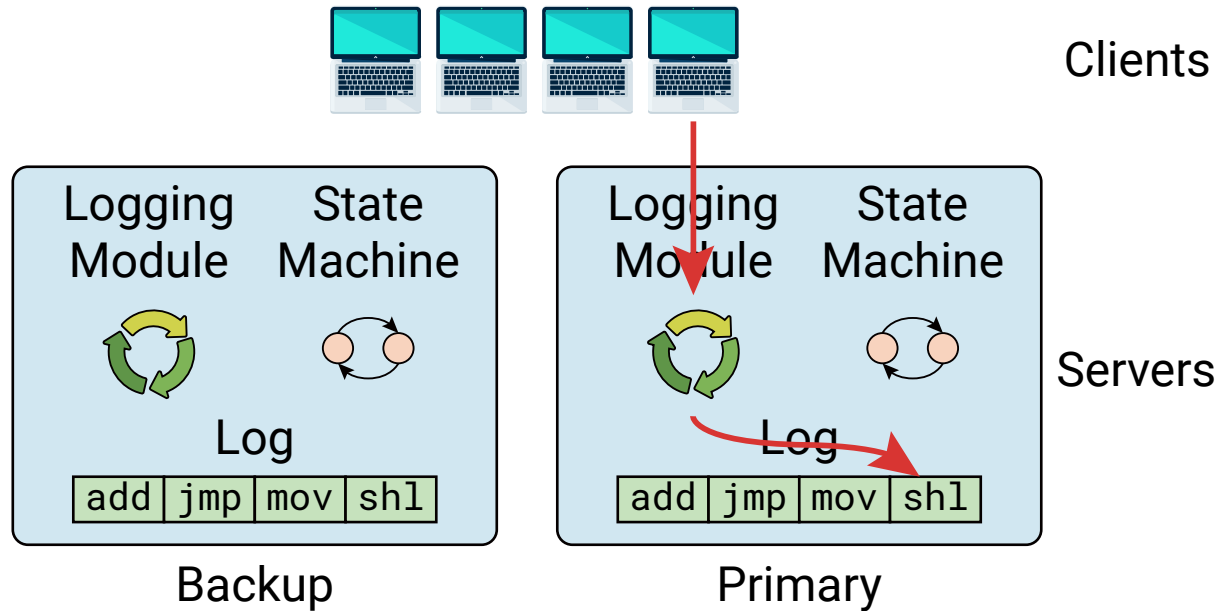
- Replicated log -> replicated state machine
 - All servers execute same operations in same order
 - All servers end up in same state

Why Does This Work?



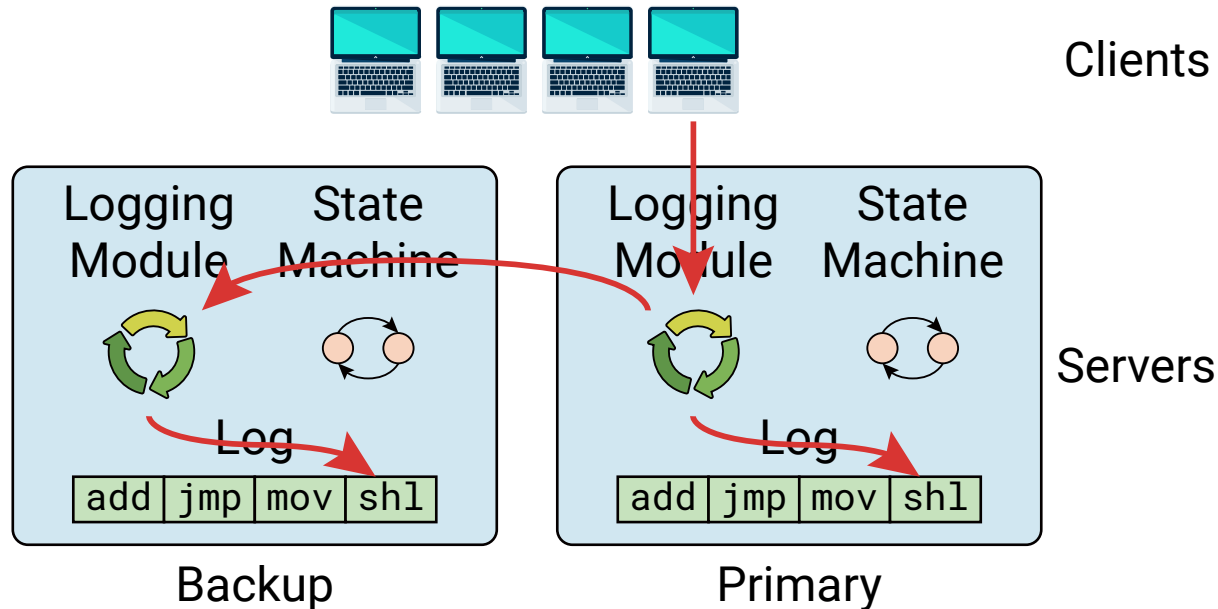
- Replicated log -> replicated state machine
 - All servers execute same operations in same order
 - All servers end up in same state

Why Does This Work?



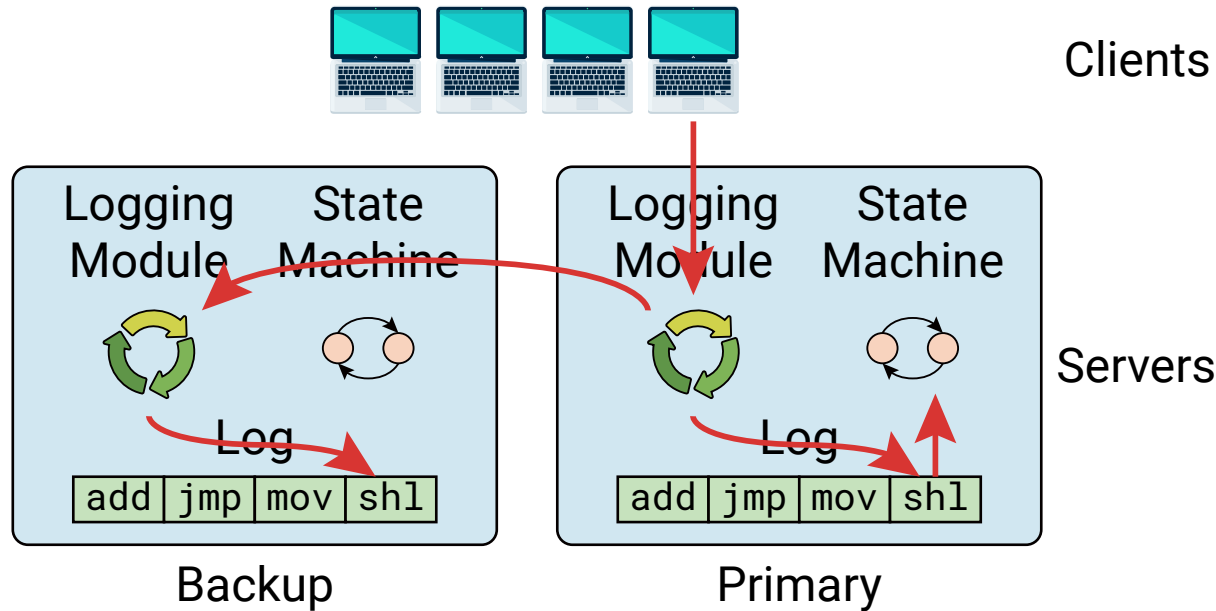
- Replicated log -> replicated state machine
 - All servers execute same operations in same order
 - All servers end up in same state

Why Does This Work?



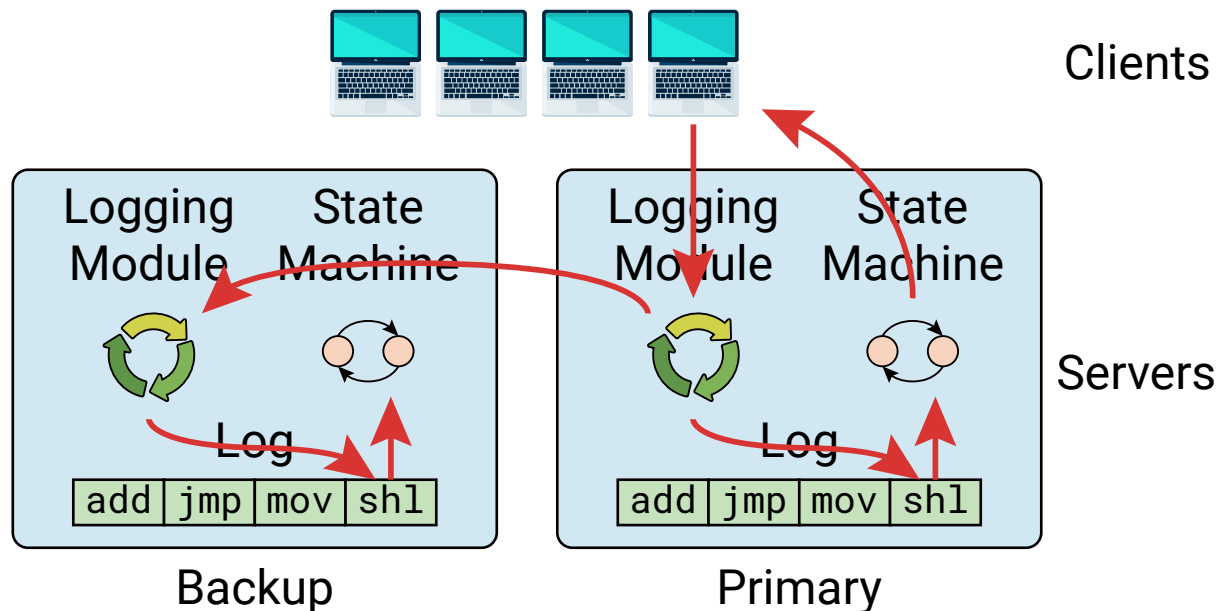
- Replicated log -> replicated state machine
 - All servers execute same operations in same order
 - All servers end up in same state

Why Does This Work?



- Replicated log -> replicated state machine
 - All servers execute same operations in same order
 - All servers end up in same state

Why Does This Work?



- Replicated log -> replicated state machine
 - All servers execute same operations in same order
 - All servers end up in same state



Need Determinism? Make It So!



- Operations are deterministic
 - No events with ordering based on local clock
 - Convert timer, network, user into logged events
 - Nothing using random inputs
- Execution order of operations is identical
 - Most RSMs are single threaded



Example: Make **random()** Deterministic

```
random.seed(a=None)
```

Initialize the random number generator.

If `a` is omitted or `None`, the current system time is used.

If `a` is an `int`, its absolute value is used directly.

```
random.getstate()
```

Return an object capturing the current internal state of the generator. This object can be passed to `setstate()` to restore the state.

Example: Make **random()** Deterministic

- Primary:
 - Initiates PRNG with OS-supplied randomness, gets initial seed
 - Sends initial seed to backup
- Backup:
 - Initiates PRNG with seed from primary



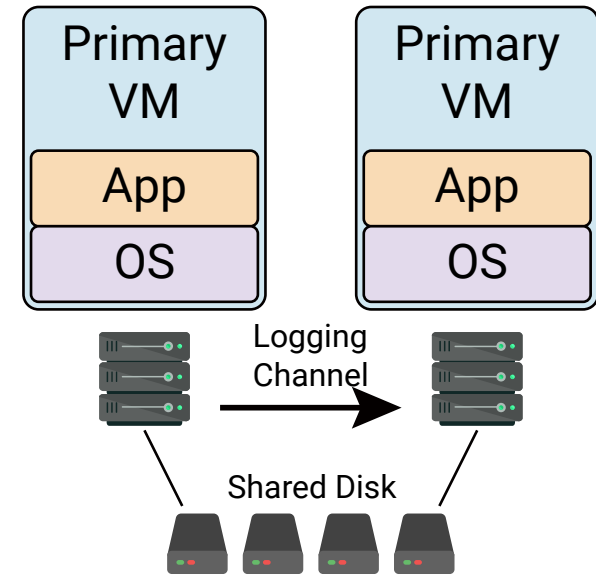
Case Study: A Practical Fault-Tolerant VM



- VMware vSphere Fault Tolerance (VM-FT)
- Goals:
 1. Replication of the **whole virtual machine**
 2. **Completely transparent** to apps and clients
 3. **High availability** for any existing software

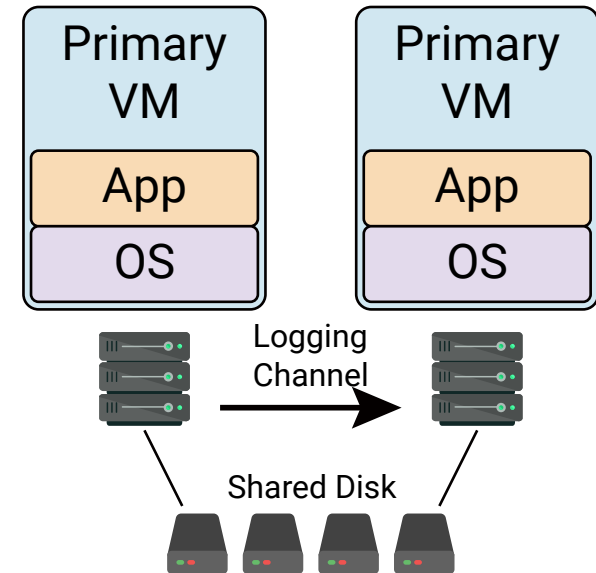
vSphere Overview

- Two virtual machines (**primary, backup**) on different bare metal
- **Logging channel** runs over network
- **Shared disk** via fiber channel



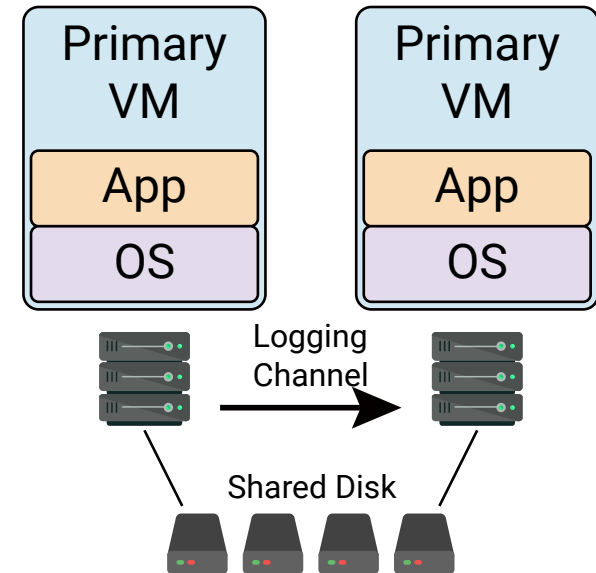
Virtual Machine I/O

- VM inputs
 - Incoming network packets
 - Disk reads
 - Keyboard and mouse events
 - Clock timer interrupt events



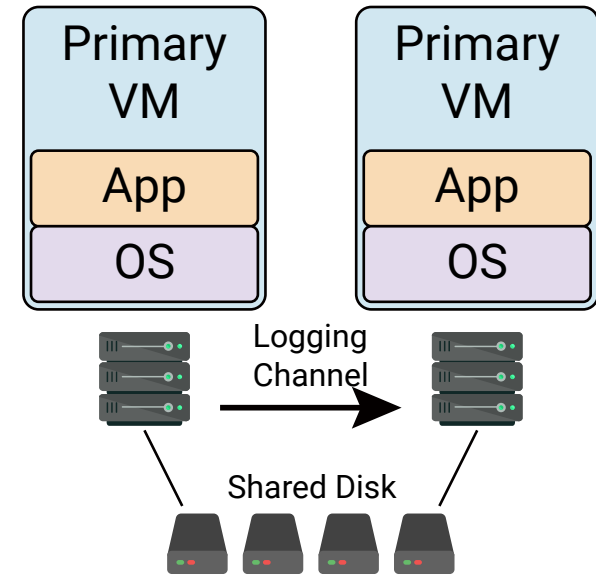
Virtual Machine I/O

- VM inputs
 - Incoming network packets
 - Disk reads
 - Keyboard and mouse events
 - Clock timer interrupt events
- VM outputs
 - Outgoing network packets
 - Disk writes



vSphere Overview

- Primary sends inputs to backup
- Backup outputs dropped
- Primary-backup heartbeats
 - If primary fails, backup takes over



VM-FT: Challenges



1. Making the backup an **exact replica of primary**
2. Making the system behave like a single server
3. Avoiding two primaries (Split Brain)

Log-based VM Replication

- **Step 1:** Hypervisor at primary logs causes of non-determinism
 1. Log results of **input events**
 - Including current program counter value for each
 2. Log results of **non-deterministic instructions**
 - e.g. log result of timestamp counter read

Log-based VM Replication

- **Step 1:** Hypervisor at primary logs causes of non-determinism
 1. Log results of **input events**
 - Including current program counter value for each
 2. Log results of **non-deterministic instructions**
 - e.g. log result of timestamp counter read
- **Step 2:** Primary hypervisor sends log entries to backup
 - Backup hypervisor **replays** the log entries
 - Stops backup VM at next input event or non-deterministic instruction
 - Delivers **same input** as primary
 - Delivers **same non-deterministic instruction result** as primary

VM-FT Challenges

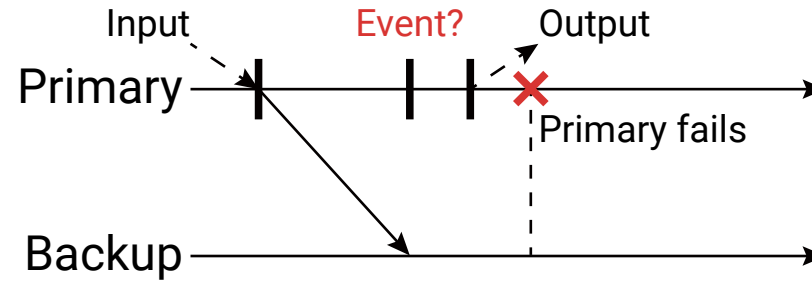


1. Making the backup an exact replica of primary
2. Making the system behave like a single server
 - **Fault Tolerance Protocol**
3. Avoiding two primaries (Split Brain)

Primary to Backup Failover

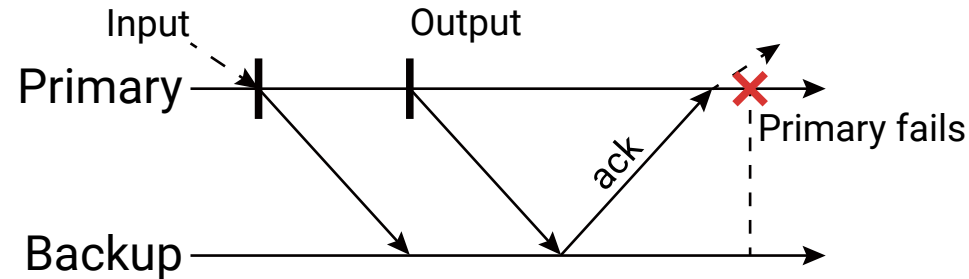
- When backup takes over, non-determinism makes it **execute differently** than primary would have
 - This is okay!
- Output requirement
 - When backup takes over, execution is **consistent** with **outputs** the primary has already sent

The Problem of Inconsistency



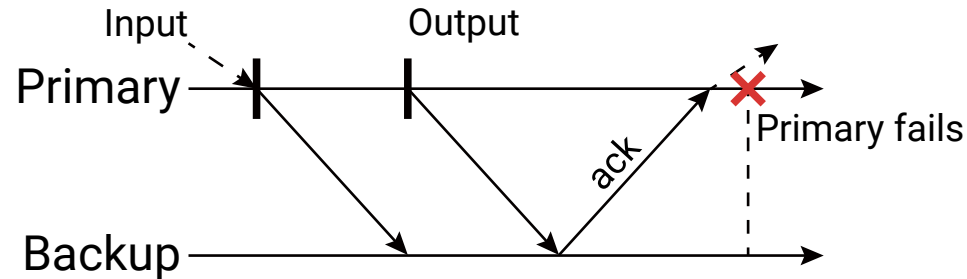
VM-FT Protocol

- Primary logs each output operation
- Delays sending output until backup acknowledges it
 - But does not need to delay execution



VM-FT Protocol

- Primary logs each output operation
- Delays sending output until backup acknowledges it
 - But does not need to delay execution



Restart execution at last output event



VM-FT Backup Promotion Process



- Backup switches from backup mode to primary mode at the last output event in the log
- Should it resend the last output event (packet)?



VM-FT Backup Promotion Process



- Backup switches from backup mode to primary mode at the last output event in the log
- Should it resend the last output event (packet)?
 - Not sending same as the network dropping it
 - Okay to not send
 - Resending same as sending a duplicate packet, which TCP does
 - Okay to send
 - Paper doesn't specify

VM-FT Challenges



1. Making the backup an exact replica of primary
2. Making the system behave like a single server
3. Avoiding two primaries (Split Brain)
 - Logging channel may **break**

▶ Detecting and Responding to Failures

- Primary and backup each run UDP heartbeats, monitor logging traffic from their peer
- Before “going live” execute **atomic test-and-set** on variable in shared storage
- If the replica finds variable already set, it **fails itself**



Backup Promotion Additional Work



- Advertise same MAC address as primary was using
- Startup a new backup
- If the primary detects the backup failed
 - It starts a new backup

Primary-Backup Replication

- State-machine replication: same input, deterministic processing -> identical replicas
- VM-FT: we can do this with whole machines!
 - Determinism tricky, but doable
 - Primary delays output until acked by backup
 - Use atomic test-and-set on shared disk to avoid split brain

© 2026 James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

All rights reserved.

These lecture materials are intended solely for educational use in the course *Distributed Systems*. Permission is granted to students enrolled in the course to reproduce these materials, in whole or in part, for personal, non-commercial use only (including printing and photocopying). Any further reproduction or distribution, including but not limited to public posting on websites, social media, forums, or file-sharing platforms, distribution to non-enrolled individuals, or any commercial use, is prohibited without the prior written permission of the author.

This slide deck is adapted from Princeton University's *Distributed Systems* course materials, originally created by Michael Freedman, Kyle Jamieson, and Wyatt Lloyd, and licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0) License. The original authors and source are gratefully acknowledged.